

Programming and Data Structures Laboratory (CS19003)

Assignment 8 (Section 6): 2D array

Department of CSE, IIT Kharagpur

1st April, 2026

General instructions to be followed strictly:

- Name your file as `<roll_no>_<assignment_no>_<question_no>`. For example, if your roll number is 14CS10001 and you are submitting assignment 3 question 1, then name your file as `14CS10001_3_1.c`
- Write your name, roll number, and assignment number at the beginning of your program.
- Use proper indentation in your code and comment. Marks may be deducted without proper indentation and comments.
- Submitting the **correct file** is your responsibility, any request of re-submission after the lab will **never** be entertained.
- **Zero tolerance** policy will be strictly followed for **plagiarism**. The marks will be put straight zero if such cases are identified.

Question-1 (25 points)

Write a C program that processes a grayscale image represented as a dynamically allocated two-dimensional array. The image has size $m \times n$, where m and n are provided by the user at runtime. Each element of the matrix represents a pixel intensity value in the range 0 to 255. The program should allocate memory dynamically using pointers, read all pixel values from the user, and display the image in matrix form.

Next, the program should ask the user to input a threshold value T and convert the grayscale image into a binary image. In this binary image, each pixel is assigned the value 1 if its intensity is greater than or equal to T , and 0 otherwise. The resulting binary matrix should then be displayed clearly. This step simulates identifying “bright” regions in the image.

The main task is to determine the largest rectangular region consisting entirely of 1s in the binary image. The rectangle must be aligned with the rows and columns of the matrix. The program should examine different possible rectangular regions and compute their areas, where the area is defined as the number of pixels inside the rectangle. Finally, the program should output the maximum such area found.

Hints: While solving the problem, you may find it useful to fix two rows at a time (a top row and a bottom row) and then check which columns contain only 1s between these two rows. For such valid columns, you can count how many consecutive columns form a continuous block and compute the rectangular area using height (number of rows between top and bottom) and width (number of consecutive valid columns). Repeat this process for all possible pairs of rows and keep track of the maximum area found. Be careful with array indices and boundary conditions, and use only loops and conditional statements.

Ensure that all dynamically allocated memory is properly freed before the program terminates, and avoid using any static array declarations. Marks will be deducted for not freeing the memory.

Example:

(Input)

Enter the number of rows and columns: 4 5

Enter the image as matrix:

```
10 200 210 5 0
50 220 230 10 5
0 0 240 250 10
5 10 20 30 40
```

Enter the threshold T: 200

(Output)

Binary Image:

```
0 1 1 0 0
0 1 1 0 0
0 0 1 1 0
0 0 0 0 0
```

Largest Rectangle Area: 4

Question-2 (25 points)

Write a C program that performs image filtering on a grayscale image represented as a dynamically allocated two-dimensional array. The image has size $m \times n$, where m and n are provided at runtime. Each element of the matrix represents a pixel intensity value between 0 and 255. The program should allocate memory dynamically, read all pixel values from the user, and display the original image.

The program should then ask the user to input an odd integer k (such as 3 or 5), followed by a $k \times k$ matrix of integers representing a kernel (filter). For each pixel, place the kernel centered at that pixel and compute a weighted sum of overlapping elements. Multiply each image value with the corresponding kernel value and add all such products. The final value is obtained by dividing this sum by the total sum of the kernel elements (consider only the valid overlapping elements when near boundaries). Store the result in a new dynamically allocated matrix and display the filtered image.

To understand the process, consider the following example of a 5×5 image and a 3×3 kernel:

$$\text{Image} = \begin{bmatrix} 10 & 20 & 30 & 40 & 50 \\ 15 & 25 & 35 & 45 & 55 \\ 20 & 30 & 100 & 50 & 60 \\ 25 & 35 & 45 & 55 & 65 \\ 30 & 40 & 50 & 60 & 70 \end{bmatrix}$$
$$\text{Kernel} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

For the center pixel (value 100), compute the weighted sum using the 3×3 neighborhood:

$$\begin{aligned} & (25 \cdot 1) + (35 \cdot 2) + (45 \cdot 1) \\ & + (30 \cdot 2) + (100 \cdot 4) + (50 \cdot 2) \\ & + (35 \cdot 1) + (45 \cdot 2) + (55 \cdot 1) = 880 \end{aligned}$$

The sum of kernel elements is:

$$1 + 2 + 1 + 2 + 4 + 2 + 1 + 2 + 1 = 16$$

So the new value is:

$$880/16 = 55$$

For a boundary pixel such as the top-left element (10), only valid overlapping elements are considered:

$$(10 \cdot 4) + (20 \cdot 2) + (15 \cdot 2) + (25 \cdot 1) = 135$$

$$\text{Kernel sum (valid part)} = 4 + 2 + 2 + 1 = 9$$

$$135/9 = 15$$

Ensure that all dynamically allocated memory is properly freed before the program terminates, and avoid using any static array declarations. Marks will be deducted for not freeing the memory.

Example:

(Input)

Enter the image size:

5 5

Enter the image matrix:

10 20 30 40 50

15 25 35 45 55

20 30 100 50 60

25 35 45 55 65

30 40 50 60 70

Enter the kernel size: 3

Enter the kernel matrix:

1 2 1

2 4 2

1 2 1

(Output)

Filtered Image:

15 22 32 42 47

18 26 38 47 53

23 33 55 52 57

28 36 46 56 62

30 38 48 58 65